



Лекция 2

Коллекции. Unit-тестирование.

Maven

Колекції

Колекціями називають структури, що призначені для зберігання *однотипних даних*.

Всі колекції Java призначені для зберігання нащадків класу **Object**.

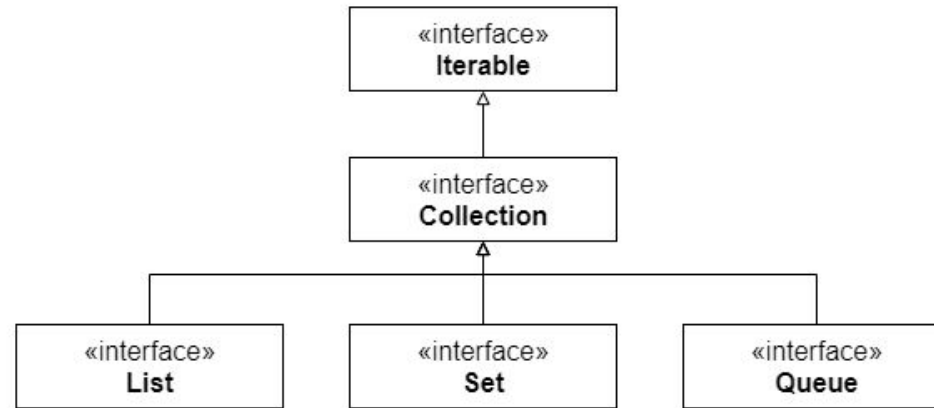
- **Типізовані колекції** дозволяють обмежити потрапляння об'єктів невідповідного типу до колекції на етапі компіляції.

Два основних інтерфейси - два принципово різних різновиду колекцій:

Collection – група об'єктів;

Map – асоційований масив об'єктів.

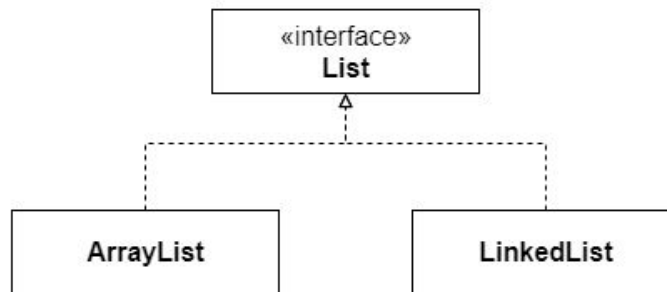
Collection



Основні методи інтерфейсу Collection

```
boolean add (Object o);  
boolean addAll(Collection c);  
int size();  
void clear();  
boolean contains(Object o);  
boolean containsAll(Collection c);  
boolean isEmpty();  
Iterator iterator();  
boolean remove(Object o);  
boolean removeAll(Collection c);  
Stream<E> stream();  
Object[] toArray();
```

Lists



```
List<Integer> list = new ArrayList<>();  
list.add(10);
```

Об'єкти цієї колекції зберігаються в порядку додавання або залежно від індекса, за яким було додано об'єкт.

Основне значення для цієї колекції має індекс, за допомогою індексу об'єкти додаються, видаляються та добуваються.

ArrayList

ArrayList – являє собою швидке перебирання колекції і швидкий доступ до її елементів за індексом.

Якщо довжини внутрішнього масиву не вистачає для додавання нового об'єкту, всередині класу створюється новий масив більшого обсягу, і всі елементи попереднього масиву копіюються в новий.

При створенні **ArrayList** варто зазначати його розмір, якщо він хоча б приблизно відомий (це запобігатиме зайвим операціям копіювання елементів масиву)

```
List<Integer> list = new ArrayList<>(20);
```

LinkedList

Елементи в **LinkedList** впорядковані за індексом, і останній доданий у колекцію елемент займає останню позицію в ній. Дані в такій колекції зберігаються у вигляді двобічно зв'язаного списку.

Доступ до елементів за індексом **get(int index)** призводить до перебирання елементів колекції починаючи з першого і до **index**.

LinkedList додатково імплементує **java.util.Queue**, за рахунок чого колекція має додаткові методи.

У **LinkedList** наявна низка методів, що не входять до інтерфейсу **List**:

- **addFirst()** та **addLast()** - додавання на початок та в кінець списку;
- **removeFirst()** та **removeLast()** - видалити перший та останній елементи;
- **getFirst()** та **getLast()** - отримати перший та останній елементи.

Доступ до елементів списків за індексом

```
List<Integer> list = List.of(1, 2, 3, 4, 5);  
for (int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i));  
}
```

Для навігації у **LinkedList** при великій кількості об'єктів використання доступу за індексом **є неефективним!**

forEach та Iterator

```
List<Integer> list = List.of(1, 2, 3, 4, 5);
```

```
// цикл foreach
```

```
for (Integer value : list) {  
    System.out.println(value);  
}
```

```
// або так
```

```
list.forEach(System.out::println);
```

**А що якщо ми хочемо видалити певні елементи під час проходження?
(видалити всі парні числа)**

```
// цикл з Iterator
```

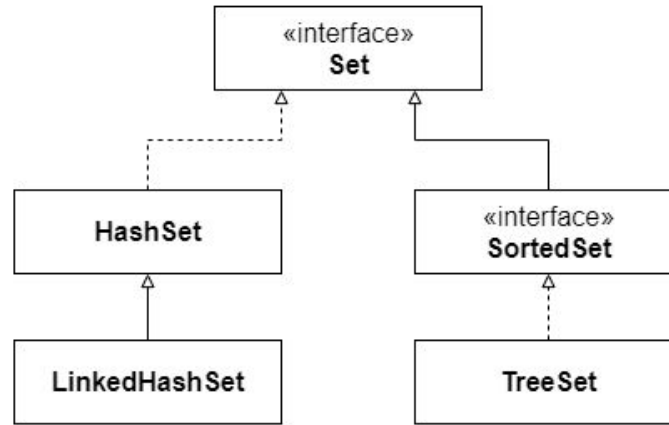
```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));  
for (Iterator<Integer> iter = list.iterator(); iter.hasNext();) {  
    Integer value = iter.next();  
    // якщо залишок ділення на 2 дорівнює 0  
    if (value % 2 == 0) {  
        iter.remove();  
    }  
}
```

ConcurrentModificationException

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));  
for (Integer value : list) {  
    if (value == 4) {  
        list.add(6);  
    }  
}
```

Exception in thread "main" java.util.ConcurrentModificationException
 at java.base/java.util.ArrayList\$Itr.checkForComodification(ArrayList.java:1013)
 at java.base/java.util.ArrayList\$Itr.next(ArrayList.java:967)
 at Test.main(Test.java:9)

Sets



```
Set<String> set = new LinkedHashSet<>();  
set.add("abc");
```

Інтерфейс **Set** зберігає множину об'єктів, **що не повторюються**.

Додавання повторюваних об'єктів до Set не провокує виникнення виключень, але такі об'єкти не потрапляють у множину.

Для проходження по множині використовується ітератор

```
Set<Integer> set = new HashSet<>();  
set.add(1);  
set.add(3);  
set.add(2);  
System.out.println(set);  
set.add(1);  
set.add(4);  
set.add(3);  
System.out.println(set);
```

[1, 2, 3]

[1, 2, 3, 4]

HashSet

HashSet – невідсортована та невпорядкована колекція, для вставки елементу використовуються методи **hashCode()** та **equals(...)**.

Чим ефективніше реалізований метод **hashCode()**, тим ефективніше працює колекція.

HashSet використовується у випадках, коли не важливий порядок елементів в колекції, але важливою є унікальність елементів в ній.

LinkedHashSet

За функціональністю **LinkedHashSet** схожий на **HashSet**, але на відміну від **HashSet**, елементи в колекції впорядковані.

Так само, як і для **HashSet**, при використанні **LinkedHashSet** необхідно перевизначити метод **hashCode()**.

TreeSet

TreeSet - колекція, що дозволяє зберігати елементи у відсортованому вигляді.

При додаванні нового об'єкту він стає на свої місце відповідно до його порядку у множині.

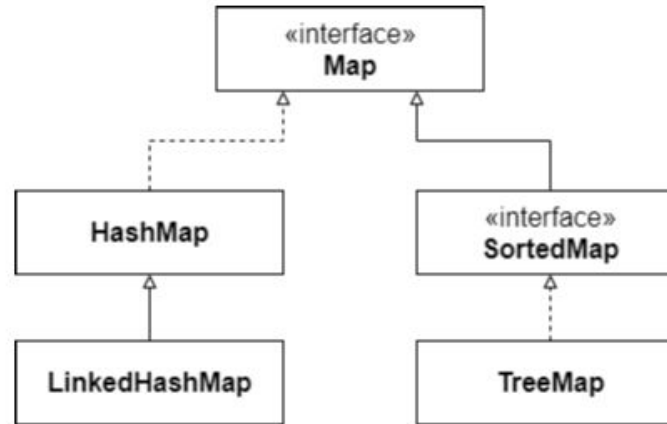
```
Set<Integer> sorted = new TreeSet<>();  
sorted.add(2);  
sorted.add(3);  
sorted.add(1);  
System.out.println(sorted); // Буде виведено [1, 2, 3]
```

Для класів, об'єкти яких будуть використовуватись в колекції, необхідно реалізувати інтерфейс **Comparable** або **Comparator**.

Інакше при додаванні елементів в колекцію буде виникати помилка:

```
java.lang.ClassCastException: class cannot be cast to class  
java.lang.Comparable
```

Map



```
Map<String, Integer> map = new HashMap<>();
map.put("one", 1);
map.put("two", 2);
```


Map

Інтерфейс **Map** часто називають асоціативним масивом.

Map здійснює відображення (mapping) множини ключів на множину значень. Таким чином об'єкти зберігаються в ньому у вигляді пар <ключ, значення>.

Map дозволяє отримати значення за ключем.

У **Map** не може бути двох пар з однаковим ключем.

Методи Map

public void put(Object key, Object value) - додає нову пару <ключ, значення>;

public Object get(Object key) – повертає value за заданим ключем або **null**, якщо нічого не знайдено;

public Set keySet() – повертає множину ключів;

boolean containsKey(Object key) – повертає **true**, якщо **Map** містить пару із заданим ключем.

Класи HashMap та LinkedHashMap

HashMap формує неупорядковану множину ключів.

Для зберігання ключів у **HashMap** та **LinkedHashMap** використовується механізм хешування.

Ключі у **HashMap** зберігаються у довільному порядку.

LinkedHashMap містить ключі в порядку їх додавання.

Приклад використання HashMap

```
Map<String, String> map = new HashMap<>();
```

```
// Заповнити її чим-небудь
```

```
map.put("one", "111");
```

```
map.put("two", "222");
```

```
map.put("three", "333");
```

```
map.put("four", "333");
```

```
// Отримати та вивести всі ключі
```

```
System.out.println("Set of keys: " + map.keySet());
```

```
// Отримати та вивести значення за ключем
```

```
String val = map.get("one");
```

```
System.out.println("one=" + val);
```

```
// Отримати та вивести всі значення
```

```
System.out.println("Collection of values: " + map.values());
```

```
// Отримати і вивести всі пари
```

```
System.out.println("Set of entries: " + map.entrySet());
```

Внутрішній інтерфейс Entry

Інтерфейс **Map.Entry** дозволяє працювати з об'єктом, який являє собою пару <ключ, значення>.

Інтерфейс містить такі методи:

boolean equals(Object o) - перевіряє еквівалентність двох пар;

Object getKey() – повертає ключ пари;

Object getValue() – повертає значення пари;

Object setValue(Object value) – змінює значення пари.

Проходження по всім **Entry**

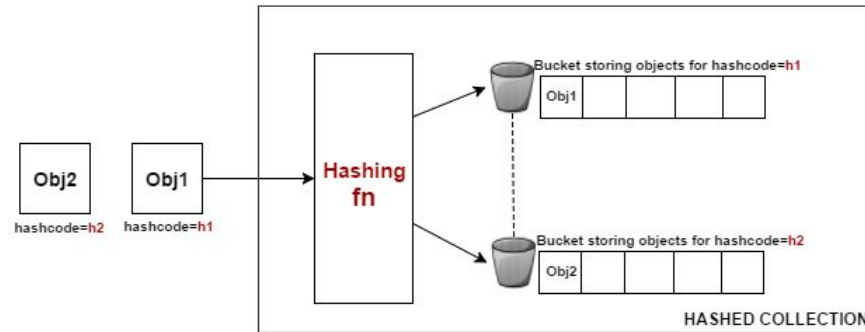
```
Map<String, Integer> map = new LinkedHashMap<>();
map.put("one", 1);
map.put("two", 2);
// ...
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + "=" + entry.getValue());
}
```

Механізм хешування

Хешування – такий спосіб зберігання та доступу до даних, при якому простір об'єктів відображається на простір адрес.

Для обчислення адреси використовується метод **hashCode()**.

За сформованою адресою знаходиться послідовність елементів. В межах цієї послідовності здійснюється пошук за допомогою методу **equals()**.



How hashing stores objects in buckets in hashed collections

Copyright © JavaBrahman.com, all rights reserved.



Впорядкованість та відсортованість

Сортування задає певний порядок розташування елементів в контейнері - залежно від правил сортування (**TreeSet**, **TreeMap**).

При впорядкуванні за сортуванням порядок вставлення елементів не важливий, для обрахунку позиції використовується інформація з об'єкту, оскільки значення має лише стан об'єкту.

String, **Integer** – впорядковуються за зростанням.

Основний спосіб порівняння об'єктів Comparable

```
public class Point implements Comparable<Point> {  
    private int x, y;  
  
    public int getDistance() {  
        return (int) Math.sqrt(x * x + y * y);  
    }  
  
    @Override  
    public int compareTo(Point o) {  
        return Integer.compare(this.getDistance(), o.getDistance());  
    }  
  
    public static void main(String[] a){  
        Set<Point> points = new TreeSet<>();  
        points.add(new Point(5, 4));  
        points.add(new Point(3, 2));  
        points.add(new Point(99, 12));  
        System.out.println(points);  
    }  
}
```


Comparator - альтернативний спосіб порівняння

```
public class PointByXComparator implements Comparator<Point> {  
    @Override  
    public int compare(Point p1, Point p2) {  
        return Integer.compare(p1.getX(), p2.getX());  
    }  
  
    public static void main(String[] args) {  
        Set<Point> points = new TreeSet<>(new PointByXComparator());  
        points.add(new Point(3, 1));  
        points.add(new Point(1, 4));  
        points.add(new Point(99, 12));  
        System.out.println(points);  
    }  
}
```

Comparator у вигляді lambda

```
public class Point {  
    private int x, y;  
  
    public static Comparator<Point> BY_X_COMPARATOR =  
        Comparator.comparing(Point::getX);  
  
    ...  
  
    public static void main(String[] a){  
        Set points = new TreeSet(Point.BY_X_COMPARATOR);  
        points.add(new Point(5, 4));  
        points.add(new Point(3, 2));  
        points.add(new Point(99, 12));  
        System.out.println(points);  
    }  
}
```

Швидкі способи створення колекцій

Пустий List

```
Collections.emptyList()
```

Set з одним об'єктом

```
Collections.singleton(myObj)
```

List з одним об'єктом

```
Collections.singletonList(myObj)
```

List з декількома об'єктами

```
List.of(myObj1, myObj2, myObj3)
```

Пустий об'єкт Map

```
Collections.emptyMap()
```

Map з однією парою ключ-значення

```
Collections.singletonMap(key, value);
```

Map з декількома парами ключ-значення

```
Map.of(key1, value1,  
        key2, value2);
```

Synchronized and Concurrent collections

У Collections Framework більшість колекцій не синхронізовано. Окрім застарілих, таких як **Vector**.

Для того, щоб зробити синхронізовану колекцію, можна застосувати методи класу **Collections**:

```
List synchronizedList(List list)
```

```
Map synchronizedMap(Map m)
```

```
Set synchronizedSet(Set s)
```

Але краще використовувати concurrent версії колекцій, такі як

```
CopyOnWriteArrayList
```

```
ConcurrentHashMap
```

```
LinkedBlockingQueue
```

Більш детально можна почитати, наприклад, тут

<https://www.baeldung.com/java-synchronized-collections>

Stream API

Починаючи з JDK 8 в Java з'явився новий API - **Stream API**. Його завдання - спростити роботу з наборами даних, зокрема, спростити операції фільтрування, сортування та інші маніпуляції з даними. Уся основна функціональність даного API сконцентрована у пакеті **java.util.stream**.

Ключовим поняттям у Stream API є **потік даних**. Загалом термін “потік” є доволі перевантаженим в програмуванні в цілому і в Java зокрема. Стосовно Stream API потік являє собою канал передачі даних із джерела даних. Причому в якості джерела даних можуть виступати як файли так і масиви, колекції.

Однією із виразних рис Stream API є застосування лямбда-виразів, що дозволяють значно скоротити запис дій, що виконуються.

Optional

Optional - контейнер (обгортка) для значень, що може безпечно містити `null`.

```
// generate random value from 0 to 1
int randomZeroOrOne = new Random().nextInt(2);

// value буде або "str", або null
String value = randomZeroOrOne == 1 ? "str" : null;
System.out.println("Value: %s".formatted(value));

Optional<String> opt = Optional.ofNullable(value);

// isPresent
System.out.println("Present: %b".formatted(opt.isPresent()));

// map and orElse
String upperCase = opt
    .map(String::toUpperCase)
    .orElse(null);
System.out.println("Upper case: %s".formatted(upperCase));

System.out.print("Print if exists: ");
opt.ifPresent(System.out::println);
```

Більш детально <https://www.baeldung.com/java-optional>

Способи створення Stream

Створення стріма з колекції

```
Collection<String> collection = Arrays.asList("aaa", "ccc", "bb");  
Stream<String> streamFromCollection = collection.stream();
```

Створення стріма зі значень

```
Stream<String> streamFromValues = Stream.of("a1", "a2", "a3");
```

Створення стріма з масиву

```
String[] array = {"a1", "a2", "a3"};  
Stream<String> streamFromArray = Arrays.stream(array);
```

Методи роботи

Java Stream API пропонує два види методів:

- **Конвеєрні (проміжні)** — повертаєть інший stream;
- **Термінальні** — повертають інший об'єкт такий, як колекція, примітив, об'єкт, **Optional** тощо.

У стріма може бути скільки завгодно конвеєрних викликів і лише один термінальний в кінці. При цьому всі конвеєрні методи виконуватимуться “ліниво”, і доки не буде викликаний термінальний метод, ніякі дії не почнуться.

Конвеєрні методи (деякі)

filter - фільтрує записи та повертає тільки ті записи, що відповідають умові фільтрування.

```
collection.stream()  
    .filter("a1"::equals)  
    .count();
```

map - перетворює кожний з елементів стріма

```
collection.stream()  
    .map(s -> s + "_1")  
    .toList();
```

distinct - повертає стрім без дублікатів (перевірка на рівність методом equals).

```
collection.stream()  
    .distinct()  
    .collect(Collectors.toList());
```

Конвеєрні методи

peek - повертає той же стрім, але застосовує функцію до кожного з елементів стріма

```
collection.stream()  
    .map(String::toUpperCase)  
    .peek(e -> System.out.print(", " + e))  
    .collect(Collectors.toList());
```

limit - дозволяє обмежити вибірку визначеною кількістю перших елементів

```
collection.stream()  
    .limit(2)  
    .collect(Collectors.toList());
```

sorted - дозволяє сортувати значення в натуральному порядку або ж в порядку, заданому Comparator

```
collection.stream()  
    .sorted()  
    .collect(Collectors.toList());
```

Термінальні методи

collect – представлення результатів у вигляді колекцій та інших структур даних.

```
collection.stream()  
    .filter(s -> s.contains("1"))  
    .collect(Collectors.toList());
```

count – повертає кількість елементів в стрімі.

```
collection.stream()  
    .filter("str"::equals)  
    .count();
```

findFirst - повертає перший елемент зі стріма
(повертає Optional)

```
collection.stream()  
    .findFirst()  
    .orElse("1");
```

Collectors

Collectors.toList()

Collectors.toSet()

Collectors.toMap(..)

```
Map<Integer, String> map = List.of(1, 2, 3, 4, 5).stream()
    .collect(Collectors.toMap(
        // key function
        Function.identity(),
        // value function
        i -> "%s_".formatted(i)
    ));
```

Collectors.joining(..)

```
String all = List.of(1, 2, 3, 4, 5).stream()
    .map(String::valueOf)
    .collect(Collectors.joining(" "));
```

Streams (приклад)

```
int distancesSum = points.stream()  
    .filter(p -> p.getY() > 0)  
    .mapToInt(Point::getDistance)  
    .sum();
```

Unit-тестування

JUnit 

Види автоматизованих тестів

Unit-тести - перевіряють логіку роботи невеликих фрагментів коду програми, як правило, окремих методів.

Integration тести - перевіряють логіку роботи і взаємодії декількох шарів програми.

- наприклад, метод сервіса, що вносить зміни до бази даних.
- або емулюють виклики rest api

UI-тести - перевіряють роботу інтерфейсу користувача.

Given-When-Then

Given-When-Then - стиль написання тестів, специфікація поведінки системи.

Ідея полягає в тому, щоб розбити написання тестового сценарію на три розділи:

- **Дано (Given)** — стан, до того, як ви почнете описувати поведінку. Можна розглядати як попередню умову тесту.
- **Коли (When)** — поведінка, яку ви описуєте.
- **Тоді (Then)** — зміни, що ви очікуєте від поведінки.

Тести повинні бути незалежними один від одного!

Результати одного тесту не повинні впливати на інші тести. Вони повинні працювати незалежно від порядку і кількості їх запуску!

Тестові методи помічаються анотацією **@Test**

@Test

```
public void testPlus() {  
    // given  
    MyCalculator calc = new MyCalculator();  
  
    // when  
    Integer result = calc.plus(2, 5);  
  
    // then  
    assertEquals(7, result);  
}
```

Операції Assertions.assertXxx

`assertEquals(Object expected, Object actual)`

`assertNull(Object actual)`

`assertNotNull(Object actual)`

`assertTrue(boolean condition)`

`assertArrayEquals(Object[] expected, Object[] actual)`

`fail()`

etc.

Перевірка exception

@Test

```
public void testPlus_boundaries() {  
    MyCalculator calc = new MyCalculator();  
  
    IllegalArgumentException exception =  
        assertThrows(IllegalArgumentException.class, () -> {  
            calc.plus(Integer.MAX_VALUE, Integer.MAX_VALUE);  
        }, "Allowed overflow");  
  
    assertEquals(  
        "Input parameters are too big",  
        exception.getMessage());  
}
```

AssertJ style

@Test

```
public void testPlus_negative() {  
    MyCalculator calc = new MyCalculator();  
  
    Integer result = calc.plus(2, -5);  
  
    assertThat(result)  
        .as("Negative check didn't pass")  
        .isEqualTo(-3);  
}
```

Анотації Before, After, BeforeClass, AfterClass

Фікстура – це тестове оточення. Фікстури є важливою складовою тестування. Їх основним завданням є підготовка оточення з попередньо визначеним/відомим станом з метою гарантування повторюваності процесу тестування.

Методи, що позначені анотаціями **@Before** і **@After** викликаються перед та після викликом кожного тестового методу відповідно.

Статичні методи позначені анотаціями **@BeforeClass** і **@AfterClass** викликаються перед викликом першого тестового методу та після виклику останнього тестового методу відповідно.

Приклади

```
public class MyClassTest {

    @BeforeAll
    public static void setUpBeforeClass() throws Exception {
        System.out.println("setUpBeforeClass");
    }

    @AfterAll
    public static void tearDownAfterClass() throws Exception {
        System.out.println("tearDownAfterClass");
    }

    @BeforeEach
    public void setUp() throws Exception {
        System.out.println("setUp");
    }

    @AfterEach
    public void tearDown() throws Exception {
        System.out.println("tearDown");
    }

    @Test
    public void testDiv() {}

    @Test
    public void testDiv2() {}
}
```

Виведення на
екран:

```
setUpBeforeClass
setUp
tearDown
setUp
tearDown
setUpAfterClass
```



Mockito – це бібліотека для створення mock-об'єктів у Java. Mock-об'єкт – це звичайний об'єкт, всі методи якого повертають значення за замовчуванням. Використання mock-об'єктів має три великих переваги:

- дозволяє в один рядок створити екземпляр будь-якого класу або інтерфейсу, при цьому немає необхідності реалізовувати чи перевизначати його методи. Є одне виключення - final класи. Проте є інструменти які дозволяють “замокати” і їх;
- mock-об'єкт дозволяє дізнатися чи був викликаний будь-який метод у цього об'єкту, а також скільки разів та з якими параметрами такий метод був викликаний;
- mock-об'єкт можна налаштувати таким чином, щоб він реагував на виклик визначеного методу потрібним чином (повертав визначений результат, викликав виключення).

Mockito verify

```
@ExtendWith(MockitoExtension.class)
public class MyCalculatorTest {

    @Mock
    CalcResultRepository calcResultRepository;

    MyCalculator calc;

    @BeforeEach
    public void before() {
        this.calc = new MyCalculator(calcResultRepository);
    }

    @Test
    public void testPlus() {
        // when
        Integer result = calc.plus(2, 5);

        // then
        assertEquals(7, result);

        Mockito.verify(calcResultRepository)
            .saveResult(new CalcResult(List.of(2, 5), 7));
    }
}
```


Testing

1. <https://www.baeldung.com/junit>
2. <https://www.baeldung.com/mockito-argumentcaptor>
3. <https://www.baeldung.com/introduction-to-assertj>
4. <https://objectpartners.com/2013/09/18/the-benefits-of-using-assertthat-over-other-assert-methods-in-unit-tests/>



Apache Maven

Apache Maven — фреймворк для автоматизації збірки проектів на основі опису їх структури в файлах мовою POM (англ. Project Object Model), що є підмножиною XML. Використовує декларативний стиль.

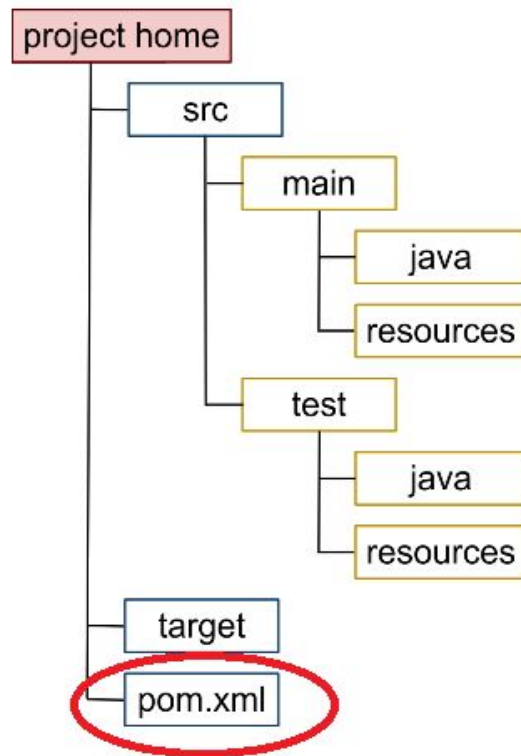
Maven Central <https://mvnrepository.com/>

Структура проекту

Конфігурація
проекті/модулі
файлі pom.xml

Maven
описується

В
В



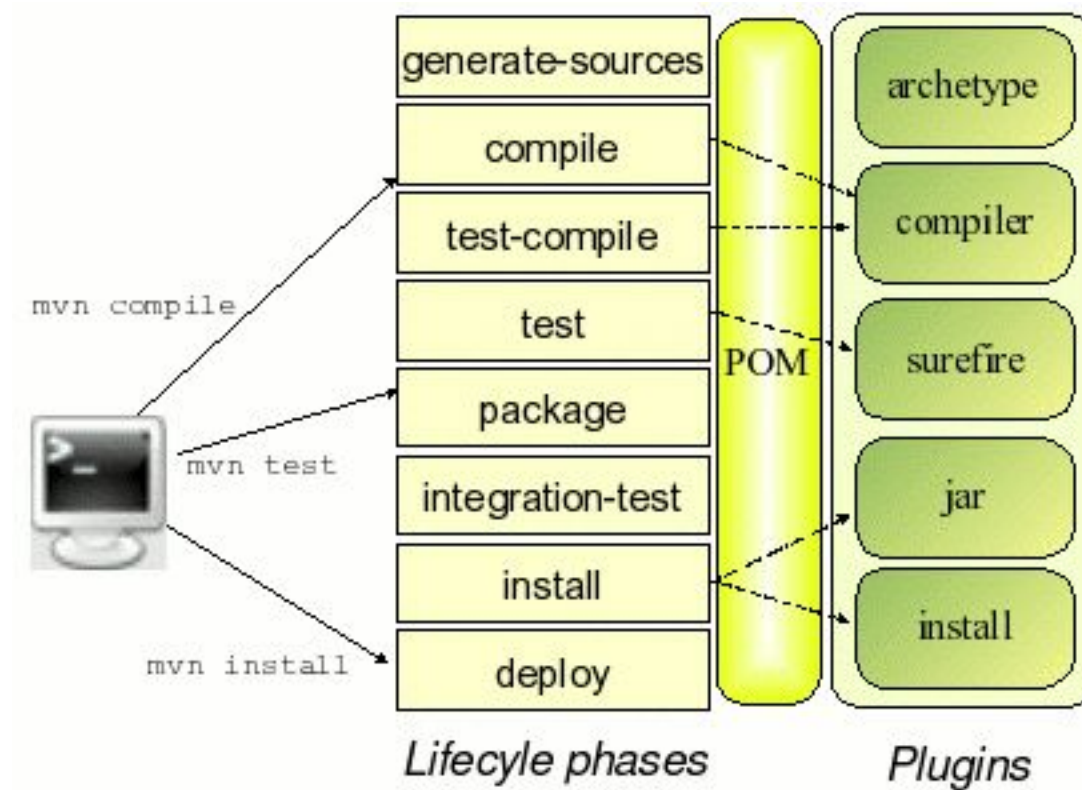
Переваги

- Повторюваність збірки;
- Стандартизація збірки;
- Декларативність збірки;
- Автоматизація процесу збірки;
- Модульність;
- Управління залежностями;
- Розширюваність плагінами.

Основні фази збірки проекту

1. **compile** - компілювання проекту;
2. **test** - тестування за допомогою JUnit тестів;
3. **package** - створення файлу .jar, war, ear в залежності від типу проекту;
4. **integration-test** - запуск інтеграційних тестів;
5. **install** - копіювання .jar (war, ear) в локальний репозиторій;
6. **deploy** - публікація файлу у відділений репозиторій.

Основні фази збірки проекту



GAV-параметри

Параметри артефакту **GAV** включають **groupId**, **artifactId**, **version**.

- **groupId** - це ID групи проекту. Найчастіше, це унікальна організація або проект;
- **artifactId** - це ідентифікатор самого проекту. Частіше всього – його ім'я;
- **version** - версія проекту.

```
<dependency>  
  <groupId>org.apache.commons</groupId>  
  <artifactId>commons-lang3</artifactId>  
  <version>3.12.0</version>  
  <scope>compile</scope>  
</dependency>
```

Область дії залежності

Область дії залежності **scope** визначає етап життєвого циклу проекту, в якому ця залежність буде використовуватися. Maven використовує 6 областей:

- **compile** - область за замовчуванням;
- **provided** - дуже схоже на compile, але ця залежність не потрапить до збірки;
- **runtime** - залежність з даною областю видимості не обов'язкова для compilation і використовується у фазі виконання;
- **test** - залежність використовується при тестуванні коду програми;
- **system** - область схожа на provided за виключенням того, що необхідно визначити фізичне розташування артефакту на диску. Артефакт з такою областю видимості maven не шукає в репозиторії;
- **import** - ця область використовується в залежності секції в <dependencyManagement> при складних зв'язках.

pom.xml з тестами

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>lk2</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>5.9.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.assertj</groupId>
      <artifactId>assertj-core</artifactId>
      <version>3.23.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>4.8.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-junit-jupiter</artifactId>
      <version>4.6.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>1.18.24</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
</project>
```



Дякуємо за увагу!